

Student 14 **JAY ADHITHIYA R** 192321048RA

4 / 5 submitted

Q1. Smart Document Editor using String, StringBuilder and StringBuffer

CODE

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("Enter Paragraph:");
        String para = sc.nextLine();

        int characters = para.length();
        int words = para.trim().split("\\s+").length;
        int sentences = para.split("[.!?]").length;

        System.out.println("Enter word to replace:");
        String oldWord = sc.nextLine();

        System.out.println("Enter new word:");
        String newWord = sc.nextLine();

        String replaced = para.replace(oldWord, newWord);

        StringBuffer history = new StringBuffer();
        history.append("Original: ").append(para).append("\n");
        history.append("After Replace: ").append(replaced).append("\n");

        String[] s = replaced.split("<= [.!?]");
        StringBuilder finalDoc = new StringBuilder();
```

```

history.append("After Replace: ").append(replaced).append("\n");

String[] s = replaced.split("(?<=[.!?])");
StringBuilder finalDoc = new StringBuilder();

for (int i = 0; i < s.length; i++) {
    StringBuilder temp = new StringBuilder(s[i].trim());
    finalDoc.append(temp.reverse()).append(" ");
}

System.out.println("Characters: " + characters);
System.out.println("Words: " + words);
System.out.println("Sentences: " + sentences);
System.out.println("Editing History:");
System.out.println(history);
System.out.println("Final Document:");
System.out.println(finalDoc);
}
}

```

INPUT

Java use strign menthod
Java
Python

OUTPUT

(no output)

Q2. Hospital Token Management System using List Interface

CODE

```
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {

        List<Patient> patientList = new ArrayList<>();

        patientList.add(new Patient(101, "Alice", "Cardiology", 45));
        patientList.add(new Patient(102, "Bob", "Neurology", 30));
        patientList.add(new Patient(103, "Charlie", "Cardiology", 50));
        patientList.add(new Patient(104, "David", "Orthopedics", 28));

        System.out.println("--- Registered Patients (Registration Order) ---");
        for (Patient p : patientList) {
            System.out.println(p);
        }

        int updateId = 102;
        for (Patient p : patientList) {
            if (p.id == updateId) {
                p.name = "Bob Smith";
                p.age = 32;
                break;
            }
        }

        int cancelId = 103;
        patientList.removeIf(p -> p.id == cancelId);

        String targetDept = "Cardiology";
        System.out.println("\n--- Patients in " + targetDept + " Department ---");
        for (Patient p : patientList) {
            if (p.department.equalsIgnoreCase(targetDept)) {
```

```

        patientList.removeIf(p -> p.isIn(targetDept));

        String targetDept = "Cardiology";
        System.out.println("\n--- Patients in " + targetDept + " Department ---");
        for (Patient p : patientList) {
            if (p.department.equalsIgnoreCase(targetDept)) {
                System.out.println(p);
            }
        }

        System.out.println("\n--- First and Last Registered Patients ---");
        if (!patientList.isEmpty()) {
            System.out.println("First Registered Patient: " + patientList.get(0));
            System.out.println("Last Registered Patient : " + patientList.get(patientList.size() - 1));
        }

        System.out.println("\n--- Final Patient List ---");
        for (Patient p : patientList) {
            System.out.println(p);
        }
    }
}

class Patient {
    int id;
    String name;
    String department;
    int age;

    public Patient(int id, String name, String department, int age) {
        this.id = id;
        this.name = name;
        this.department = department;
    }
}

```

Report

```
        this.age = age;
    }

    public String toString() {
        return "ID: " + id + "\nName: " + name + "\nDepartment: " + department + "\nAge: " + age;
    }
}
```

OUTPUT

(no output)

Q3. Airport Boarding Queue Management using Queue Interface

CODE

```
import java.util.*;
class Main {
    public static void main(String[] args) {
        Queue<String> boardingQueue = new LinkedList<>();
        PriorityQueue<String> vipQueue = new PriorityQueue<>();
        boardingQueue.add("Rahul");
        boardingQueue.add("Anitha");
        boardingQueue.add("Karan");
        boardingQueue.add("Priya");

        vipQueue.add("Arun");
        vipQueue.add("Bala");
        vipQueue.add("Charan");

        System.out.println("Normal Boarding Queue:");
        System.out.println(boardingQueue);
        System.out.println("\nVIP Queue:");
        System.out.println(vipQueue);
        System.out.println("\nNext Normal Passenger: " + boardingQueue.peek());
    }
}
```

```

boardingQueue.add("Karan");
boardingQueue.add("Priya");

vipQueue.add("Arun");
vipQueue.add("Bala");
vipQueue.add("Charan");

System.out.println("Normal Boarding Queue:");
System.out.println(boardingQueue);
System.out.println("\nVIP Queue:");
System.out.println(vipQueue);
System.out.println("\nNext Normal Passenger: " + boardingQueue.peek());
System.out.println("Next VIP Passenger: " + vipQueue.peek());
System.out.println("\nBoarding VIP Passengers:");
while (!vipQueue.isEmpty()) {
    System.out.println("Boarded: " + vipQueue.poll());
    System.out.println("Remaining VIP Queue: " + vipQueue);
}
System.out.println("\nBoarding Normal Passengers:");
while (!boardingQueue.isEmpty()) {
    System.out.println("Boarded: " + boardingQueue.poll());
    System.out.println("Remaining Boarding Queue: " + boardingQueue);
}
System.out.println("\nAll passengers have boarded.");
}
}

```

OUTPUT

(no output)

```

        adding newId = sc.next();
        boolean added = certificateSet.add(newId);
        if (added) {
            System.out.println("Certificate ID added successfully.");
        } else {
            System.out.println("Duplicate entry! Certificate ID already exists.");
        }
        break;

    case 2:
        System.out.print("Enter Certificate ID to verify: ");
        String checkId = sc.next();
        if (certificateSet.contains(checkId)) {
            System.out.println("Certificate ID exists.");
        } else {
            System.out.println("Certificate ID does not exist.");
        }
        break;

    case 3:
        System.out.println("Certificates in Insertion Order:");
        if (certificateSet.isEmpty()) {
            System.out.println("No certificates found.");
        } else {
            for (String id : certificateSet) {
                System.out.println(id);
            }
        }
        break;

    case 4:
        System.out.println("Certificates in Sorted Order:");
        if (certificateSet.isEmpty()) {
            System.out.println("No certificates found.");
        } else {
            Set<String> sortedSet = new TreeSet<>(certificateSet);
            for (String id : sortedSet) {
                System.out.println(id);
            }
        }
    }
}

```

```
        case 5:
            System.out.println("Total number of unique certificates issued: " + certificateSet.size());
            break;

        case 6:
            System.out.print("Enter Certificate ID to remove (revoke): ");
            String removeId = sc.next();
            boolean removed = certificateSet.remove(removeId);
            if (removed) {
                System.out.println("Certificate revoked and removed successfully.");
            } else {
                System.out.println("Certificate ID not found.");
            }
            break;

        case 7:
            System.out.println("Exiting program. Goodbye!");
            break;

        default:
            System.out.println("Invalid choice! Please try again.");
    }
} while (choice != 7);
}
```

OUTPUT (no output)

Q5. Smart Inventory Management using Map Interface

— Not Submitted —